AI Project Part B Report
Cascade!

1 Overview

Tasked with making an AI agent to play cascade, we decided on implementing iterative deepening minimax(with α-β pruning), along with a myriad of algorithmic optimizations for our agents' decision making. Throughout the weeks, it has been a mixture of figuring out game relevant strategies, speeding up the search algorithm, and playing loads of Cascade!

2 Agent Design

At the heart of our agent is how it selects its actions... what search algorithm it uses, and — due to the propensity of games like cascade to have a huge range of different actions — how exactly we manage to evaluate different board states.

2.1 The Search algorithm

Cascade is a deterministic and a perfect-information type game with lots of tactical play one can make, so naturally we gravitated towards minimax. It offered us the ability to tweak the behavior of our agent through the evaluation function, based on what we(not cascade grandmasters by any means) ascertained were positions an excellent player would like to be in. Therefore, ruling out other algorithms like Monte Carlo search....

2.2 Enhancements on Standard MiniMax

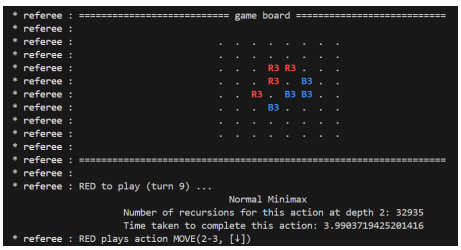
Throughout coding the first version of minimax, which had random placements and a really simple evaluation function to start off with:

$$\text{EVAL}(s) = \text{our tokens } - \text{enemy tokens}$$

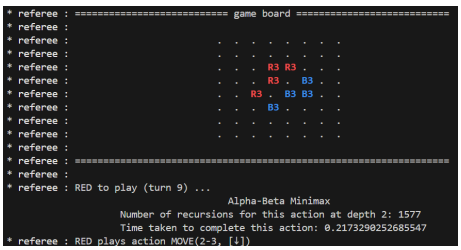
it became clear that standard minimax just wont cut it, as the maximum depth we found it reasonably go to was 2. Modifications on plain minimax seemed due....

2.2.1 α-β pruning

Cascade and a large branching factor go hand in hand, especially deeper into the tree. Too many moves just aren't promising, so why bother looking deeper into that branch? Alpha beta pruning offers a way to speed up the search process and enable us to look deeper into the tree with the extra time available to us.
In figure 1 we can see that indeed alpha beta pruning does have a huge impact, and now we can go to depth 4 without too much trouble.



(a) normal minimax



(b) minimax with alpha beta pruning

Figure 1: (a) and (b) at depth 2 shows how many recursions happens for a single action and the corresponding time taken

2.2.2 Iterative deepening

Next we had to address the time constraint, each move needed to have a budget based on how much time we had left.... with our current approach it didn't matter how long an action would go for, it will keep going until it reaches the depth limit. That's not great since with larger DEPTH_LIM it almost always goes over the time limit.

Iterative deepening gave us the ability to first try depth_lim(not a constant anymore) 1, then 2, then 3 and so on until the allocated budget for this action is exceeded, and if it does we can just take the best move the previous iteration found. An additional bonus we found because of this: with less tokens and less stacks towards the end of the game, our agent was able to reach a much larger depth due to the lower branching factor.

2.3 The evaluation function

Now we had to step back and look at our evaluation function. Our agent was not great, it chose random placements with no thought behind its consequences, and in the play phase it spread its pieces all over the board and wouldn't engage the enemy, just played safe moves.

2.3.1 Placement evaluation function

Playing Cascade, placement plays a big role on which side is winning. Out of all legal placements which one should our agent aim for?

We recognized that time is of the essence and in the placement phase we don't need to be looking 4 moves ahead, so we opted for a greedy strategy to placement based on this evaluation function:

$$\text{EVAL}_p(s) = 2.0 \cdot \text{center\_score}(s) + 0.8 \cdot \text{coverage}(s) + \text{support}(s) + \text{opp\_pressure}(s)$$

- ***center_score***: Prefer stacks to be closer to the center, central stacks have the ability to cascade off enemy stacks and prevent it self from being cascaded off in one move as well.
- ***coverage***: If stack is placed and cascaded to each adjacent square, count the sum of cells it covers. This also enforces the center-good behavior because we have more coverage, but also helps in that we space them out evenly around the center.
- ***support***: Reward placing stacks near existing ones, allows for the potential of merging and larger control of the board with the higher tower
- ***opp_pressure***: reward for placing stacks near enemy, stacks placed near enemy naturally limit their movement

*Weights were tuned manually, in that we chose them so each term contributes a comparable magnitude, preventing any single feature from dominating.

2.3.2 Play phase Evaluation Function

Here is where our agent, once it finishes its look ahead evaluates its current state, and so we address the problem with our agent and why it doesn't engage the enemy. We had to reward strategic plays.

To make sure our evaluation is actually capturing the correct aspects that matter in game quickly, we had to experiment with lots of greedy agents with different heuristics. It starts off with one agent, try out a new feature, have it go against agent without it and see if it gets any better.

$$\begin{aligned}\text{EVAL}(s) = 2.0 \cdot \text{token\_diff}(s) + \ \text{height\_penalty}(s) + \ 0.3 \cdot \text{control\_diff}(s) + \ \text{edge\_pressure}(s) \\ + 1.5 \cdot \text{threats}(s) + \ \text{pairing}(s) + \ 0.6 \cdot \text{cascade\_line\_bonus}(s) + \ \text{coord\_bonus}(s) + \ \varepsilon\end{aligned}$$

where $\varepsilon \sim \mathcal{U}(-0.05, 0.05)$

- ***token_diff***: Our tokens − enemy tokens, prefer states where we have a higher token advantage, since the point of the game is to not lose all your tokens.
- ***height_penalty***: Penalise states where we have a stack taller than 6, these stacks offer no cascading benefits since the board is only 8x8 and concentrating towers into one stack means you have less strategic moves to begin with.
- ***control_diff***: cascade_reach(*our*) − cascade_reach(*opp*), where cascade_reach is the sum of num cells a cascade can reach in each adjacent direction, wider spread means we control more of the board and restrict possible moves our opponent makes.
- ***edge_pressure***: reward our central stacks, while also penalizing enemies central stacks. Central stacks offer more escape/attack routes, also lets us prefer states where we can pressure opponent towards the edge, limiting their mobility.
- ***threats***: our_threats − enemy_threats, where both sum, over all opposing towers, the score of their closest eligible tower targeting it: the closer the better! It incentivizes our stacks to push towards enemies, while also taking care to not lose any stacks.
- ***pairing***: Count how many of our stacks are each attacking separate enemy stacks. Pushes for each stack to spread and attack 2 different enemy towers, rather than both merging and attacking one lone tower.
- ***cascade_line_bonus***: Rewards having enemies directly in our cascade path, with a higher reward if they are close to the edge. This gives enemies less room to escape, threatening to eat their tower.
- ***coord_bonus***: Reward for double teaming a single enemy stack, guarantees a kill on that specific enemy stack
- ϵ: add a small number to break ties. Makes draws less likely to happen, if some moves have equal EVAL(s).

*Weights were tuned manually, in that we chose them so each term contributes a comparable magnitude, preventing any single feature from dominating.

3 Other Aspects

After all the changes made to the evaluation, it played decently well. However, we wanted to push a little harder and further optimize the search and allow to be much faster.

- **Killer Heuristic**: For every depth in our search we save two of the best moves, i.e, the ones that caused a beta cutoff. This means that for the same depth we can search these moves first, increasing the likely hood of pruning. [**reference**]
- **Transposition Tables**: A Cache of positions we have already been in so that if we reach the same state through a different set of moves, we can just use our previous results rather than searching again. The keys are hashes provided through zobrist hashing [**reference**], and each entry contains important information about the state: The evaluation of state, depth it was search to, a flag to indicate if the search was cut short and the best move we found at this state which is used to improve the move ordering.[**reference1**] [**reference2**]
- **Quiescence Search**: Once we reach end depth, if not in a stable state then preform an additional search on only elimation moves, so that we evaluate said state once no captures are present. Goal for this was to prevent mistakes which happens because our agent couldn't see deep enough in a volatile position.[**reference**]
- **Aspiration Window**: Instead of starting every search with a alpha-beta window of $[-\infty, \infty]$ we make use of iterative deepening in that at some depth we can just take advantage of the previous iterations evaluation since it has to be close to it, with some margin of error which we chose to be $[\text{prev\_score} - \delta, \text{prev\_score} + \delta]$ with $\delta = 30$. The basic thought was that it should increase pruning, and as fall back if it fails, we just research with the full window again [**reference**]
- **History Heuristic**: similar to a killer heuristic in that we want to order the moves, it however, looks at the entire search rather than at a specific depth. We store a dictionary where the keys are the actions that caused a beta cutoff, and the values being the score which is based on the depth(if the action in history was found deeper its better!). We then sort quiet moves based on it. [**reference**]
- **Late Move Reduction**: To save time in searching, rather than go through entire depth with moves ordered later in the list(in our case we used moves indexed ≥ 4), depth $- 2$ is where we stop the search, if that move ends up looking promising we then search with entire depth.[**reference**]

4 Performance Evaluation

To evaluate our agent and how well it does, we had it go against previous versions of the same agent. We ran 10 games with our agent as RED and 10 with BLUE.

Opponent	Red			Blue			%wins our final version got
	W	D	L	W	D	L	
RandomAgent	10	0	0	10	0	0	100
GreedyAgent	10	0	0	10	0	0	100
v1_minmax_alphabeta	7	3	0	8	2	0	75
v2_iterativeDepth	4	6	0	3	7	0	36
v3_optimizedIterativeDepth	1	9	0	1	9	0	10

Figure 2: Number of wins/loses our final agent had against opponent

- RandomAgent: an agent that chooses its moves randomly
- GreedyAgent: agent that doesn't look into the future, random placements and evaluation = difference in tokens
- v1: MiniMax agent with alpha_beta pruning, random placements and evaluation = difference in tokens.
- v2: iterative depth search with placement evaluation as in 2.3.1, evaluation with just token_diff, threats and height_penalty. Includes killer heuristic and TT tables
- v3: all optimizations, and complete evaluation as in section 2.3, it has different eval weights from the final version

5 Supporting Work

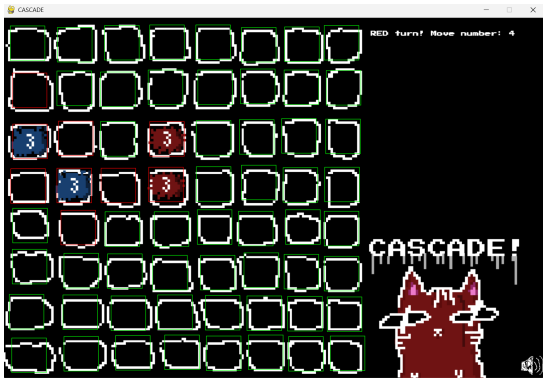
5.1 tournament.py

To enable testing of agents against each other many times, we wrote a program that ran sub processes(calling the subprocesses.run() method), to run a game, which then captures the stdout from referee to calculate number of wins each agent got.

5.2 Cascade Realised

To present the game in a more interactive way where we can play against our agents, we used the pygame library to display an actual game of cascade!

(a) cascade menu



(b) cascade game

Figure 3: CASCADE sneak peak

Try it out here: https://github.com/aalshawsh/cascade(note that it only has a humanAgent and a randomAgent nothing else, more info in README)

6 AI Tools Disclosure

In line with the subject guidelines, we want to acknowledge our use of Claude as an interactive engineering assistant throughout this project. Our use of AI was strictly limited to code analysis, debugging support, and testing infrastructure. Specifically, we utilized Claude for the following:

- **Understanding Complex Algorithms**: We used the AI during our initial planning phase to talk through and clarify the tricky mathematical edge cases of advanced search strategies. Once we had a solid grasp of the concepts, we manually built and calibrated the features ourselves.
- **Code Reviews and Quality Checks**:We used the tool to review our functional methods, helping us spot logical flaws and keep our overall codebase clean and maintainable.
- **Debugging Complex Errors**: When we hit unexpected errors, we use Claude for debugging, analyze error traces, and help us rapidly pin down the root causes.